

Trabajo Sistemas Operativos 2: parking.c

Sistemas Operativos II GIISI
USAL
Alumno: Juan Riego Vila

Índice

1. Cambios realizados el día 31 de Marzo de 2026	3
1. 1. Hora: 12:34 cambios documentados:	3
1. 2. Hora: 13:25 cambios documentados:	3
2. Cambios realizados el día 05 de Abril de 2026	5
2. 1. todos los cambios han sido realizados sobre el punto 5 del trabajo	5
2. 1. 1. Hora: 13:11 cambios documentados:	5
3. Cambios realizados el día 05 de Abril de 2026	5
3. 1. todos los cambios han sido realizados sobre el punto 5 del trabajo	5
3. 1. 1. Hora: 13:11 cambios documentados:	5
4. Cambios realizados el día 15 de Abril de 2026	5
4. 1. todos los cambios han sido realizados sobre el punto 8 del trabajo	5
4. 1. 1. Hora: 16:51 cambios documentados:	5

1. Cambios realizados el día 31 de Marzo de 2026

1. 1. Hora: 12:34 cambios documentados:

He añadido una función de ayuda para que cuando se ejecute el programa sin argumentos esta salga aclarar los argumentos que tienes que introducir para el correcto funcionamiento del programa, esta se llama «ayudaPrograma()» y hay que pasarle como argumento el parámetro argv para que utiliza el nombre del archivo.

1. 2. Hora: 13:25 cambios documentados:

He añadido la forma para capturar cuando el usuario utiliza Ctrl-C, y he formateado el código de una forma que le es más agradable al profesor para cuando lo lea y tal.

Las señales funcionan de la siguiente forma dentro de la nueva función parking la cual ahora es la principal, ya que todo en la función main suele llevar a errores entonces lo he sacado todo y metido en otra función, bien pues esa funciona ahora se llama parking, dentro de esta al principio hay un bloque de código que lo único que hace es mediante las señales que se mandan al sistema operativo capturar Ctrl-C para su manejo, nada mas ha cambiado en esta función principal a parte de la justificación del texto para que todo se pueda leer mas fácil.

```
[...]
struct sigaction sa_ctrlc;
memset(&sa_ctrlc, 0, sizeof(sa_ctrlc));
sa_ctrlc.sa_handler = manejar_ctrlc;
sigaction(SIGINT, &sa_ctrlc, NULL);
sigaction(SIGTERM, &sa_ctrlc, NULL);
[...]
```

Luego he introducido una función llamada manejar_ctrlc(), la cual mediante la señal que hemos capturado en la función parking() cambiar el valor de la variable «terminar» la cual por ahora no sirve para nada ya que no tenemos hijos ni padres que terminar.

```
[...]
void manejar_ctrlc(int signal) {
    terminar = 1; //la funcion de esto es que cuando se ejecute el
    programa parar los bucles de creacion de los hijos cuando se reciba
    ctrl-c para limpiar bien los procesos
    system("tput cnorm"); //esto lo que hace es volver a poner el
    cursor "normal", ya que cuando se ejecuta el programa a veces el
    cursor se queda en modo "escondido" esto lo que hace es cambiarle a
    modo mostar
} //fin funcion manejar_ctrlc
[...]
```

Y luego he introducido una parte del código que llama a “system” mediante la librería «stdlib.h» gracias a esto se puede recuperar el cursor ya que a veces cuando “matabas” al

proceso en medio de la ejecución este te “ocultaba” el cursor en la terminal, esta misma línea también ha sido agregada al final de la función parking para lo mismo.

```
[...]  
system("tput cnorm"); //esto lo que hace es volver a poner el  
cursor "normal", ya que cuando se ejecuta el programa a veces el  
cursor se queda en modo "escondido" esto lo que hace es cambiarle a  
modo mostrar  
[...]
```

2. Cambios realizados el dia 05 de Abril de 2026

2. 1. todos los cambios han sido realizados sobre el punto 5 del trabajo

2. 1. 1. Hora: 13:11 cambios documentados:

Introducido un programa llamado ejecutarRedirigiendoSalidaErrores.sh el cual permite redirigir todos los errores a un archivo llamado "errores.txt" como forma predeterminada este archivo es excluido mediante el archivo .gitignore para que no suba a Github permitiendo que este más limpio el repositorio.

3. Cambios realizados el dia 05 de Abril de 2026

3. 1. todos los cambios han sido realizados sobre el punto 5 del trabajo

3. 1. 1. Hora: 13:11 cambios documentados:

Introducido un programa llamado ejecutarRedirigiendoSalidaErrores.sh el cual permite redirigir todos los errores a un archivo llamado "errores.txt" como forma predeterminada este archivo es excluido mediante el archivo .gitignore para que no suba a Github permitiendo que este más limpio el repositorio.

4. Cambios realizados el dia 15 de Abril de 2026

4. 1. todos los cambios han sido realizados sobre el punto 8 del trabajo

4. 1. 1. Hora: 16:51 cambios documentados:

He introducido las "funciones de rellamada", las cuales se necesitan para hacer este apartado, porque las pide la propia función, estas se suponen que controlan el ciclo de vida de un vehiculo una vez que este se crea y ejecuta, estas son las funciones introducidas:

```
[...]
//se ejecuta cuando la biblioteca confirma que el coche ha aparcado
void aparcar_commit(HCoche hc) {
    if (debug) printf("[DEBUG] Coche %d aparcado en algoritmo [no
implementado] (Commit)\n", hc);

    //aquí es donde levantarías el semaforo para el siguiente
coche (mp->proxaparcar)
    //semop(id_sem, ...);
} //fin funcion aparcar_commit

//se ejecuta cuando el coche quiere moverse. debe bloquearse hasta que
sea seguro
```

```

void permiso_avance(HCoche hc) {
    if (debug) printf("[DEBUG] Coche %d pidiendo permiso para
avanzar...\n", hc);
    //De momento como dice el enunciado solo mensaje
    //en una version final aqui se usarian semaforos para evitar
colisiones
}//fin funcion permiso_avance

//se ejecuta cuando la biblioteca confirma que el movimiento se ha
realizado
void permiso_avance_commit(HCoche hc) {
    if (debug) printf("[DEBUG] Coche %d ha avanzado con éxito.\n",
hc);
}//fin funcion mi_permiso_avance_commit

[...]
```

Por ahora son funciones vacías ya que solo las pide el enunciado para continuar, cuando se vaya a resolver el algoritmo que pide el trabajo ya se harán uso de estas.

Y se introdujeron cambios en esta parte del código, para que el chofer llamase a esta función, como se pide en el enunciado, se le pasan las variables y funciones que he desarrollado antes, y a parte una nueva variable **alg_aux**, la cual sirve para pasar la información a las funciones callback, esta también la pide la introduce la librería (parking.h), como una variable void, la introduzco como entera porque se espere que transporte el resultado de algún algoritmo, o el tipo de algoritmo que es (ejemplo que 1, sea un switch-case para fifo, 2 para el siguiente, etc).

```

[...]
```

// fork creó el proceso hijo

```

if (pid_chofer == 0) {
    // el hijo ignora SIGINT, solo muere cuando el buzón
desaparece

    signal(SIGINT, SIG_IGN);
    struct PARKING_mensajeBiblioteca msg;

    /*hay que pasar la información a los callbacks (como
    por ejemplo el algoritmo que sea), utilizo una
    variable para pasarla por el puntero void *datos.
    */
    int alg_aux;

    while (1) {
        // si el buzón se limpia y msgrcv falla,
salimos
        if (msgrcv(id_buzon, &msg, sizeof(msg) -
sizeof(long), 0, 0) == -1) {
            break;
        }//fin if
    }
```

```

        // imprime lo que ha llegado
        printf("[CHOFER] tipo=%ld subtipo=%ld
coche=%d\n", msg.tipo, msg.subtipo, msg.hCoche);
    }//fin while
    if (debug) fprintf(stderr, "[D-CHOFER] PID=%d
muriendo\n", getpid());

    //subtipo para representar el indice del algoritmo (0
a 3)
    alg_aux = (int)msg.subtipo;

    PARKING_aparcar(
        msg.hCoche,                // el manejador del
coche recibido
        &alg_aux,                  // datos que llegan
a los callbacks
        aparcar_commit,           // funcion de
confirmacion de aparcado
        permiso_avance,           // funcion de
control de trafico
        permiso_avance_commit     // funcion de
confirmacion de movimiento
    );//fin PARKING_aparcar

    exit(0);
} //fin if
[...]
```